

Code-Layout, Readability and Reusability

Date	12 August, 2026
Team ID	Individual
Project Name	AI Knowledge Retention Predictor
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1.	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Standard PEP 8 4-space indentation enforced across all Python scripts.
2.	Proper File Structure	Files and folders are logically organized	Yes	Project structured clearly into <code>app.py</code> , <code>database.py</code> , <code>models/</code> , and <code>utils/</code> .
3.	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Self-descriptive naming conventions used (e.g., <code>retention_score</code> , <code>decay_rate</code>).
4.	Function / Method Names	Functions are descriptively named	Yes	Clear action-oriented function names (e.g.,

				<code>calculate_forgetting_curve()</code> .
5.	Code Comments	Inline and block comments explain logic	Yes	Docstrings and comments detail mathematical formulas and API handling.
6.	Modular Design	Code is split into reusable functions/modules	Yes	Clear isolation of database CRUD operations, decay math engines, and UI views.
7.	No Redundant Code	Duplicate or unused code is removed	Yes	Shared utility helper functions eliminate repetitive code patterns.
8.	Error Handling	Exceptions and errors are handled gracefully	Yes	<code>try-except</code> blocks protect database transactions and API requests.

Reusable Components/Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1.	Forgetting Curve Calculator	Python / NumPy	Core retention dashboard, quiz scheduling module	High
2.	Gemini Prompt Generator	Python / Google Gemini API	Dynamic quiz engine, topic review generator	High
3.	SQLite DB Manager	Python / SQLite3	User session authentication, score persistence	High
4.	Plotly Retention Renderer	Python / Plotly	Analytics dashboard, summary reports	Medium

5.	Streamlit Custom CSS Theme	Python / CSS	Global UI rendering across all pages	Medium
----	----------------------------	--------------	--------------------------------------	--------

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5/5	Excellent separation of frontend presentation, database access, and core logic.
Readability	5/5	Adheres closely to Python standards; explicit variable and function naming throughout.
Reusability	4/5	Math and API helper modules are decoupled and easy to reuse across future projects.
Documentation / Comments	4/5	Comprehensive inline docstrings explain complex Ebbinghaus mathematical equations.
Overall Score	4.5/5	High-quality, maintainable codebase ready for production deployment.